

Introduction

Grid alignment problems show up more often in interviews than people expect, and not because anyone is testing you on satellite imagery. They're testing whether you can spot when a problem's real bottleneck is hidden data, not the grid itself. LeetCode 835, Image Overlap, looks like an image-processing problem, but it's really a lesson in reframing: instead of searching over positions, you search over relationships between the few points that actually matter.

This is also a great problem for building instinct around when to trade a "obvious" $O(n^4)$ brute force for something driven entirely by sparse data. If you've ever frozen in an interview because a solution technically works but clearly won't scale, this pattern is one of the cleanest ways to build that muscle.

Problem Overview

You're given two square binary grids of the same size, `img1` and `img2`, each filled with 0s and 1s. You're allowed to slide one grid — left, right, up, or down, by any amount — before placing it on top of the other. No rotating, no flipping, just translation.

After sliding, the "overlap" is the number of positions where both grids have a 1 at the same cell. Your job is to find the slide that produces the largest possible overlap, and return that maximum count.

The key constraint that should catch your eye: sliding never wraps around and never rotates. A slide is fully described by exactly two numbers — how far you moved vertically, and how far you moved horizontally.

Example

Imagine a small grid where `img1` has 1s scattered at a few positions, and `img2` has the same shape of 1s, just shifted down by one row and right by one column.

If you slide `img1` down by 1 and right by 1, every one of its 1-cells now sits exactly on top of a 1-cell in `img2`. The overlap count equals however many 1s were in the smaller of the two point sets.

In the actual example from the problem, sliding `img1` right by one and down by one causes three 1-cells to align perfectly. No other slide does better, so the answer is 3.

Intuition

The naive instinct is to physically slide the grid: try every possible offset, stack the grids, and count matching 1s at each position. It works, but think about what you're actually spending time on. For an $n \times n$ grid, there are roughly $O(n^2)$ possible offsets, and checking each one costs $O(n^2)$ work to compare cells. That's $O(n^4)$ total, and worse, you're spending most of that time comparing zeros against zeros — cells that could never contribute to an overlap.

The reframe: an overlap only ever happens where a 1 in `img1` lines up with a 1 in `img2`. There's no other way to get credit. So instead of asking "what happens at this shift," ask the reverse question: "for this pair of 1-cells, one from each image, what shift would align them?"

The answer is just their coordinate difference — a row gap and a column gap. That difference is a shift *vector*. If you compute this vector for every pair of 1-cells (one from each grid), the shift that appears most often is the shift that aligns the most 1s. You've turned a search over positions into a tally over votes.

This works because the number of 1s is often far smaller than n^2 , especially in sparse grids like satellite photos of land. You stop paying for every empty cell and only pay for the pixels that actually carry information.

Brute Force Solution

Idea: Slide `img2` over `img1` at every possible offset, stack them, and count matching 1s.

Algorithm: 1. For every possible row shift and column shift within the grid's bounds. 2. Overlay the shifted grid on the original. 3. Count cells where both are 1. 4. Track the maximum count seen.

Advantages: Simple to reason about, directly matches the problem's plain-English description, easy to verify by hand on small examples.

Disadvantages: Wastes enormous effort scanning offsets and cells that never contain any 1s. Doesn't exploit sparsity at all.

Complexity: Time $O(n^4)$ — roughly n^2 possible shifts, each costing $O(n^2)$ to evaluate. Space $O(n^2)$ for the grids themselves (or $O(1)$ extra if done in place).

Optimal Solution

Step 1: Walk both grids once each and collect the coordinates of every 1-cell. Call these `pts1` and `pts2`.

Step 2: For every point in `pts1` paired with every point in `pts2`, compute the row difference and column difference. This pair `(row_gap, col_gap)` is the shift vector this pairing "votes" for.

Step 3: Tally these votes in a dictionary, keyed by the `(row_gap, col_gap)` tuple.

Step 4: The answer is the highest vote count in that dictionary — the shift that aligns the most 1-cells simultaneously.

Step	What happens	Cost
Extract points	Scan grid once	$O(n^2)$
Pair and vote	Compare every pt1 to every pt2	$O(pts1 \times pts2)$
Read max	Scan dictionary values	$O(\text{distinct shifts})$

Because `|pts1|` and `|pts2|` are both at most n^2 , but in practice much smaller for sparse grids, this dramatically outperforms brute force sliding.

Python Code

```
from collections import defaultdict
from typing import List

def largest_overlap(img1: List[List[int]], img2: List[List[int]]) -> int:
    n = len(img1)

    pts1 = [(r, c) for r in range(n) for c in range(n) if img1[r][c] == 1]
    pts2 = [(r, c) for r in range(n) for c in range(n) if img2[r][c] == 1]

    if not pts1 or not pts2:
        return 0

    counts = defaultdict(int)
    for r1, c1 in pts1:
        for r2, c2 in pts2:
            shift = (r1 - r2, c1 - c2)
            counts[shift] += 1

    return max(counts.values())
```

Code Walkthrough

- `n = len(img1)` grabs the grid dimension once.
- The two list comprehensions build `pts1` and `pts2`, each a list of `(row, col)` coordinates where the grid holds a 1. This is the step that throws away every zero cell — from here on, zeros never enter the calculation.
- The early return handles the case where either image has no 1s at all: there's nothing to overlap, so the answer is immediately 0.
- `counts` is a dictionary mapping a shift vector to how many point-pairs voted for it.
- The nested loop pairs every point in `pts1` with every point in `pts2`. `(r1 - c1, c1 - c2)` ... actually `(r1 - r2, c1 - c2)` is the shift that would move the point in `img2` onto the point in `img1`.
- `counts[shift] += 1` records that vote.
- `max(counts.values())` returns the highest tally, which is the largest achievable overlap.

Dry Run

Say `pts1 = [(0, 0), (1, 1)]` and `pts2 = [(1, 0), (2, 1)]`.

Pairing `(0,0)` with `(1,0)` gives shift `(-1, 0)`. Pairing `(0,0)` with `(2,1)` gives shift `(-2, -1)`. Pairing `(1,1)` with `(1,0)` gives shift `(0, 1)`. Pairing `(1,1)` with `(2,1)` gives shift `(-1, 0)`.

Tally: `(-1, 0)` appears twice, everything else once. Max count is 2, so the answer is 2 — sliding by row -1, col 0 aligns both points.

Complexity Analysis

Time: $O(|pts1| \times |pts2|)$, bounded above by $O(n^4)$ in the worst case (a fully dense grid), but typically far smaller. This is correct because every possible overlap must come from some pair of 1-cells, and we enumerate exactly those pairs — no more, no less.

Space: $O(\text{distinct shifts})$, which is bounded by $O(|pts1| \times |pts2|)$ in the worst case but usually much smaller in practice, since many pairs collapse onto the same shift.

Alternative Solutions

FFT-based cross-correlation is the standard technique for aligning real images and would technically work here, computing overlap counts for all shifts simultaneously via convolution. But with `n` capped

at 30, the overhead of setting up FFTs isn't worth it — the point-pairing approach is simpler to implement, easier to reason about, and just as fast at this scale.

Edge Cases

- **Empty grids (all zeros):** Either `pts1` or `pts2` is empty, function returns 0 immediately.
- **Single 1 in each grid:** Only one shift vector is ever voted for; that's the answer, trivially 1.
- **Identical grids, no shift needed:** The zero shift `(0, 0)` should collect the most votes.
- **All 1s (dense grid):** Every pair votes, and the maximum shift is the "no movement" case, which yields n^2 .
- **1s on grid boundary:** Coordinates work identically at edges; no special handling needed since we never physically move the grid.

Common Mistakes

1. Trying to physically slide and re-index the grid arrays, leading to off-by-one errors or index-out-of-bounds handling for the "hanging off the edge" parts.
2. Forgetting the early return for empty point lists, causing `max()` on an empty dictionary to throw a `ValueError`.
3. Swapping the order of subtraction (`r2 - r1` instead of `r1 - r2`), which flips the direction of the shift — usually harmless for the final count but confusing if you later need the actual shift, not just its magnitude.
4. Using a nested list or a 2D array to store vote counts instead of a dictionary, wasting memory over the full shift-space instead of only tracking shifts that actually occurred.
5. Recomputing the point lists inside the nested loop instead of extracting them once beforehand.

Interview Questions

- How would you modify this to return the actual best shift, not just the overlap count?
- What happens to the time complexity if the grids aren't square?
- Could you solve this using FFT-based convolution instead, and what's the tradeoff?
- How would your approach change if grids could also be rotated, not just slid?
- What's the worst-case input that maximizes the runtime of this algorithm?

Similar Problems

- **LeetCode 30 (Substring with Concatenation of All Words):** similar "reduce to relevant positions" reframing.
- **LeetCode 149 (Max Points on a Line):** also relies on tallying vectors/slopes between point pairs in a hash map.
- **LeetCode 1878 (Get Biggest Three Rhombus Sums):** grid-coordinate reasoning with geometric constraints.
- **LeetCode space-transform problems** in general benefit from the same "convert positions to relative differences" trick.

Key Takeaways

When a grid problem seems dominated by sparse "interesting" cells, stop thinking about positions and start thinking about relationships between those cells. Converting a position search into a vote tally on relative offsets is a pattern worth recognizing — it shows up anywhere alignment, matching, or translation invariance matters.

Watch the Video

Prefer seeing this traced out visually with the actual grid diagrams and two live quizzes to test your understanding? Watch the full walkthrough here: {LINK}

About the Series

This is part of Fun with Learning Technology, a series that breaks down LeetCode problems the way you'd actually want them explained in a real interview prep session — building intuition first, writing clean production-quality Python second, and never skipping the "why" behind an approach.

Call To Action

If this helped the vote-on-vectors trick click for you, the video has the full visual walkthrough — give it a like on YouTube, subscribe for more daily LeetCode breakdowns, and consider subscribing to the Substack for deeper dives. Drop a comment with your own approach or a problem you'd like to see covered next, and share this with anyone prepping for coding interviews.

Quick Review

Overlapping grids with translations only — what's the tell?

When you only need shift amounts (dx,dy), not rotation, think: collect coordinates, count vector differences via hashmap.

Time complexity of the vector-tally approach for Image Overlap?

$O(n^4)$ worst case: comparing every '1' in image A against every '1' in image B.

Space complexity of the shift-vector counting approach?

$O(n^4)$ in the worst case for the hashmap of (dx,dy) counts, since each dimension ranges -n..n.

Common bug when computing shift vectors between two 1-cells?

Forgetting shift is directional (A's cell minus B's cell, not the reverse) — flips sign and silently corrupts the vote tally.

Brute-force grid shifting vs vector-tally: what's the tradeoff?

Brute force tries every shift and rescans the whole grid each time (slow); tally approach only touches actual 1s, much faster when sparse.

Interview follow-up: what if rotation were also allowed?

Vector-tally breaks down; you'd need to also enumerate rotations (4 or 8 orientations) and repeat the shift-vote process per orientation.